SKIN DISORDER PROJECT

- This project focuses on the analysis and classification of erythemato-squamous skin diseases using clinical and histopathological data. The dataset consists of 34 attributes out of which 33 are feature varaibles and 1 is the target variable representing the skin disease class. The features include a combination of clinical observations (such as erythema, scaling, itching) and histopathological indicators (such as acanthosis and parakeratosis).
- Number of instances: 366
- Number of features: 33
- Target variable: Skin disease class (multiclass)
- An extensive exploratory data analysis was performed to understand the data structure, distributions, and relationships among features. Multiple machine learning models, including Logistic Regression, Decision Tree, Random Forest, Support Vector Machine, and K-Nearest Neighbors, were trained and evaluated to identify the most effective predictive model. Based on comparative performance analysis, the Logistic Regression demonstrated superior accuracy and robustness. Additionally, insights derived from the analysis were used to provide practical suggestions to assist dermatologists in the early identification and diagnosis of skin diseases, thereby supporting timely clinical decision-making.

OBJECTIVE

- The objective of this project is to analyze clinical and histopathological data related to erythemato-squamous skin diseases and develop an effective machine learning–based predictive model to accurately classify different types of skin disorders. The project aims to explore the underlying patterns in the data through exploratory data analysis, compare the performance of multiple classification algorithms, and identify the most suitable model for prediction. Additionally, the project seeks to provide meaningful insights and practical suggestions to assist doctors in the early and accurate diagnosis of skin diseases, thereby improving clinical decision-making and patient outcomes.
- The following are the tasks carried out in this project:
- Task 1:-Prepare a complete data analysis report on the given data.
- Task 2:-Create a predictive model using machine learning techniques to predict the various classes of skin disease.
- Task3:-Suggestions to the Doctors to identify the skin diseases of the patient at the earliest.

CODING PART

Data Analysis (EDA)

```
#importing libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv('dataset_35_dermatology (1).csv')

df.head()
```

```
   erythema  scaling  definite_borders  itching  koebner_phenomenon  \
0         2        2                 0        3                   0
1         3        3                 3        2                   1
2         2        1                 2        3                   1
3         2        2                 2        0                   0
4         2        3                 2        2                   2

   polygonal_papules  follicular_papules  oral_mucosal_involvement  \
0                  0                   0                         0
1                  0                   0                         0
2                  3                   0                         3
3                  0                   0                         0
4                  2                   0                         2

   knee_and_elbow_involvement  scalp_involvement  ...  \
0                           1                  0  ...
1                           1                  1  ...
2                           0                  0  ...
3                           3                  2  ...
4                           0                  0  ...

   disappearance_of_the_granular_layer  \
0                                    0
1                                    0
2                                    0
3                                    3
4                                    2

   vacuolisation_and_damage_of_basal_layer  spongiosis  \
0                                        0           3
1                                        0           0
2                                        2           3
3                                        0           0
4                                        3           2
```

```
   saw-tooth_appearance_of_retes  follicular_horn_plug  \
0                              0                     0
1                              0                     0
2                              2                     0
3                              0                     0
4                              3                     0

   perifollicular_parakeratosis
inflammatory_monoluclear_inflitrate  \
0                              0                                       1

1                              0                                       1

2                              0                                       2

3                              0                                       3

4                              0                                       2


   band-like_infiltrate  Age  class
0                     0   55      2
1                     0    8      1
2                     3   26      3
3                     0   40      1
4                     3   45      3

[5 rows x 35 columns]
```

```
df.shape
```

```
(366, 35)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 366 entries, 0 to 365
Data columns (total 35 columns):
 #   Column                                     Non-Null Count  Dtype
---  ------                                     --------------  -----
 0   erythema                                   366 non-null    int64
 1   scaling                                    366 non-null    int64
 2   definite_borders                           366 non-null    int64
 3   itching                                    366 non-null    int64
 4   koebner_phenomenon                         366 non-null    int64
 5   polygonal_papules                          366 non-null    int64
 6   follicular_papules                         366 non-null    int64
 7   oral_mucosal_involvement                   366 non-null    int64
 8   knee_and_elbow_involvement                 366 non-null    int64
 9   scalp_involvement                          366 non-null    int64
```

```
 10  family_history                              366 non-null    int64
 11  melanin_incontinence                        366 non-null    int64
 12  eosinophils_in_the_infiltrate               366 non-null    int64
 13  PNL_infiltrate                              366 non-null    int64
 14  fibrosis_of_the_papillary_dermis            366 non-null    int64
 15  exocytosis                                  366 non-null    int64
 16  acanthosis                                  366 non-null    int64
 17  hyperkeratosis                              366 non-null    int64
 18  parakeratosis                               366 non-null    int64
 19  clubbing_of_the_rete_ridges                 366 non-null    int64
 20  elongation_of_the_rete_ridges               366 non-null    int64
 21  thinning_of_the_suprapapillary_epidermis    366 non-null    int64
 22  spongiform_pustule                          366 non-null    int64
 23  munro_microabcess                           366 non-null    int64
 24  focal_hypergranulosis                       366 non-null    int64
 25  disappearance_of_the_granular_layer         366 non-null    int64
 26  vacuolisation_and_damage_of_basal_layer     366 non-null    int64
 27  spongiosis                                  366 non-null    int64
 28  saw-tooth_appearance_of_retes               366 non-null    int64
 29  follicular_horn_plug                        366 non-null    int64
 30  perifollicular_parakeratosis                366 non-null    int64
 31  inflammatory_monoluclear_inflitrate         366 non-null    int64
 32  band-like_infiltrate                        366 non-null    int64
 33  Age                                         366 non-null    object
 34  class                                       366 non-null    int64
dtypes: int64(34), object(1)
memory usage: 100.2+ KB
```

```
df.dtypes
```

```
erythema                                        int64
scaling                                         int64
definite_borders                                int64
itching                                         int64
koebner_phenomenon                              int64
polygonal_papules                               int64
follicular_papules                              int64
oral_mucosal_involvement                        int64
knee_and_elbow_involvement                      int64
scalp_involvement                               int64
family_history                                  int64
melanin_incontinence                            int64
eosinophils_in_the_infiltrate                   int64
PNL_infiltrate                                  int64
fibrosis_of_the_papillary_dermis                int64
exocytosis                                      int64
acanthosis                                      int64
hyperkeratosis                                  int64
parakeratosis                                   int64
clubbing_of_the_rete_ridges                     int64
```

```
elongation_of_the_rete_ridges                int64
thinning_of_the_suprapapillary_epidermis     int64
spongiform_pustule                           int64
munro_microabcess                            int64
focal_hypergranulosis                        int64
disappearance_of_the_granular_layer          int64
vacuolisation_and_damage_of_basal_layer      int64
spongiosis                                   int64
saw-tooth_appearance_of_retes                int64
follicular_horn_plug                         int64
perifollicular_parakeratosis                 int64
inflammatory_monoluclear_inflitrate          int64
band-like_infiltrate                         int64
Age                                         object
class                                        int64
dtype: object
```

```
df['Age'] = df['Age'].replace('?', np.nan)
df['Age'] = pd.to_numeric(df['Age'], errors='coerce')

df['Age'] = df['Age'].fillna(df['Age'].median())
```

- The Age feature was initially stored as an object data type due to the presence of non-numeric values ('?'). These values were converted to NaN and the column was cast to numeric format. Median imputation was applied to handle missing vlaues, ensuring compatibility with statistical analysis and machine learning models.

```
df.dtypes
```

```
erythema                                     int64
scaling                                      int64
definite_borders                             int64
itching                                      int64
koebner_phenomenon                           int64
polygonal_papules                            int64
follicular_papules                           int64
oral_mucosal_involvement                     int64
knee_and_elbow_involvement                   int64
scalp_involvement                            int64
family_history                               int64
melanin_incontinence                         int64
eosinophils_in_the_infiltrate                int64
PNL_infiltrate                               int64
fibrosis_of_the_papillary_dermis             int64
exocytosis                                   int64
acanthosis                                   int64
hyperkeratosis                               int64
parakeratosis                                int64
clubbing_of_the_rete_ridges                  int64
elongation_of_the_rete_ridges                int64
```

```
thinning_of_the_suprapapillary_epidermis      int64
spongiform_pustule                            int64
munro_microabcess                             int64
focal_hypergranulosis                         int64
disappearance_of_the_granular_layer           int64
vacuolisation_and_damage_of_basal_layer       int64
spongiosis                                    int64
saw-tooth_appearance_of_retes                 int64
follicular_horn_plug                          int64
perifollicular_parakeratosis                  int64
inflammatory_monoluclear_inflitrate           int64
band-like_infiltrate                          int64
Age                                         float64
class                                         int64
dtype: object
```

```
df.describe()
```

```
         erythema     scaling  definite_borders     itching  \
count  366.000000  366.000000        366.000000  366.000000
mean     2.068306    1.795082          1.549180    1.366120
std      0.664753    0.701527          0.907525    1.138299
min      0.000000    0.000000          0.000000    0.000000
25%      2.000000    1.000000          1.000000    0.000000
50%      2.000000    2.000000          2.000000    1.000000
75%      2.000000    2.000000          2.000000    2.000000
max      3.000000    3.000000          3.000000    3.000000

       koebner_phenomenon  polygonal_papules  follicular_papules  \
count          366.000000         366.000000          366.000000
mean             0.633880           0.448087            0.166667
std              0.908016           0.957327            0.570588
min              0.000000           0.000000            0.000000
25%              0.000000           0.000000            0.000000
50%              0.000000           0.000000            0.000000
75%              1.000000           0.000000            0.000000
max              3.000000           3.000000            3.000000

       oral_mucosal_involvement  knee_and_elbow_involvement  \
count                 366.000000                  366.000000
mean                    0.377049                    0.614754
std                     0.834147                    0.982979
min                     0.000000                    0.000000
25%                     0.000000                    0.000000
50%                     0.000000                    0.000000
75%                     0.000000                    1.000000
max                     3.000000                    3.000000

       scalp_involvement  ...  disappearance_of_the_granular_layer  \
count         366.000000  ...                           366.000000
```

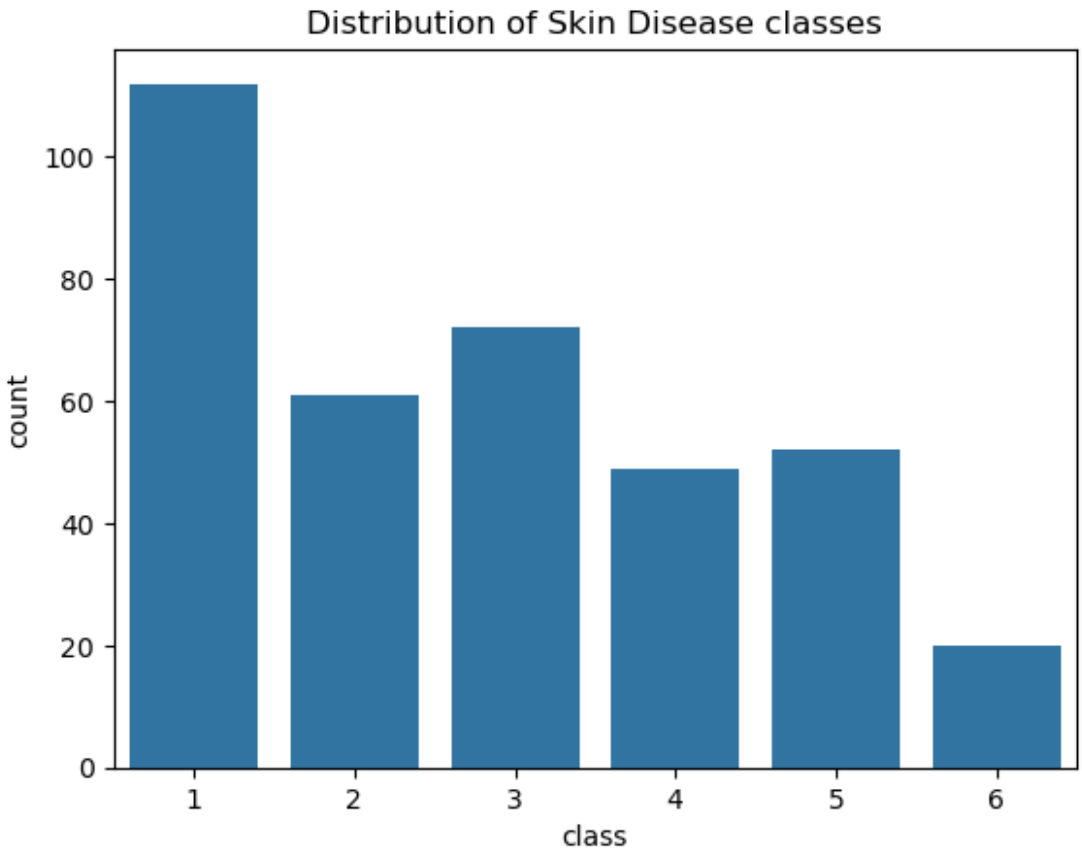
```
mean             0.519126  ...                                0.464481
std              0.905639  ...                                0.864899
min              0.000000  ...                                0.000000
25%              0.000000  ...                                0.000000
50%              0.000000  ...                                0.000000
75%              1.000000  ...                                1.000000
max              3.000000  ...                                3.000000

       vacuolisation_and_damage_of_basal_layer  spongiosis  \
count                               366.000000  366.000000
mean                                  0.456284    0.953552
std                                   0.954873    1.130172
min                                   0.000000    0.000000
25%                                   0.000000    0.000000
50%                                   0.000000    0.000000
75%                                   0.000000    2.000000
max                                   3.000000    3.000000

       saw-tooth_appearance_of_retes  follicular_horn_plug  \
count                     366.000000            366.000000
mean                        0.453552              0.103825
std                         0.954744              0.450433
min                         0.000000              0.000000
25%                         0.000000              0.000000
50%                         0.000000              0.000000
75%                         0.000000              0.000000
max                         3.000000              3.000000

       perifollicular_parakeratosis
inflammatory_monoluclear_inflitrate  \
count                     366.000000
366.000000
mean                        0.114754
1.866120
std                         0.488723
0.726108
min                         0.000000
0.000000
25%                         0.000000
1.000000
50%                         0.000000
2.000000
75%                         0.000000
2.000000
max                         3.000000
3.000000

       band-like_infiltrate         Age       class
count            366.000000  366.000000  366.000000
mean               0.554645   36.267760    2.803279
```

```
std                 1.105908   15.156875    1.597803
min                 0.000000    0.000000    1.000000
25%                 0.000000   25.000000    1.000000
50%                 0.000000   35.000000    3.000000
75%                 0.000000   48.000000    4.000000
max                 3.000000   75.000000    6.000000

[8 rows x 35 columns]
```

OBSERVATION

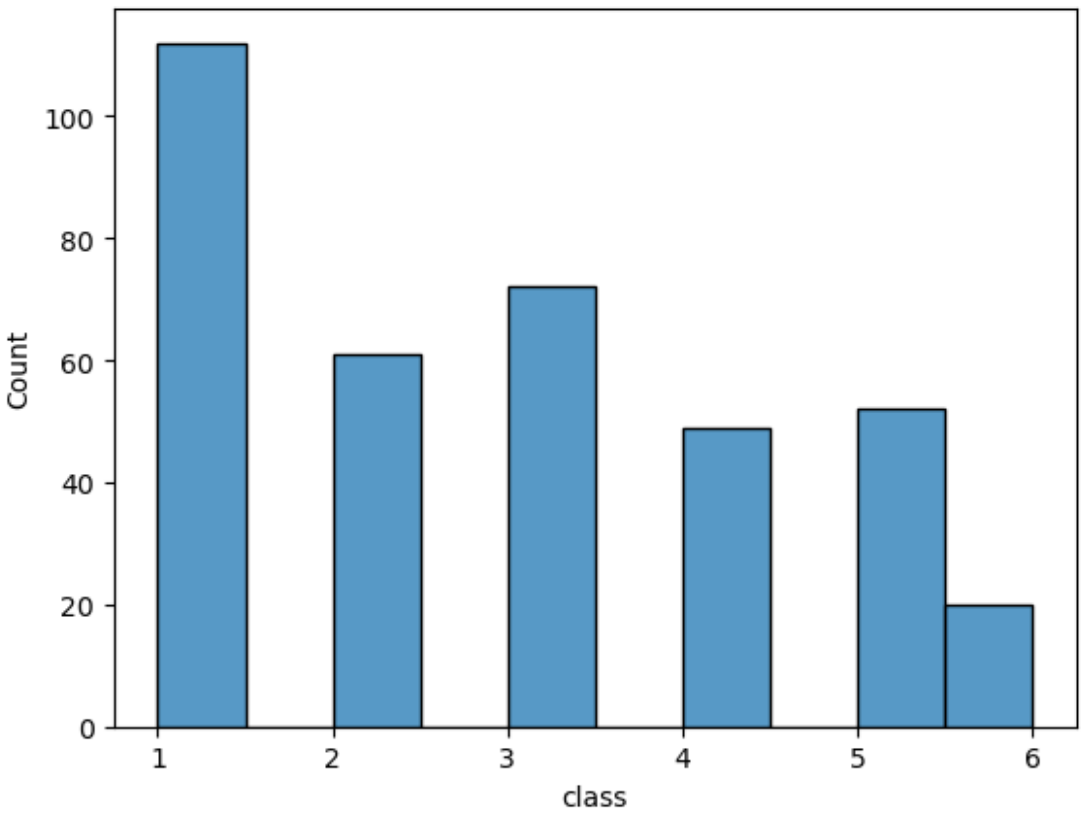
- Most clinical and histopathological attributes have values ranging from 0-3.
- It shows that the variables are categorically ordinal The mean values of erythema and scaling are relatively higher than others.
- The median values are often 0 or 1 which shows that many sympthoms are absent in atleast half of the class.
- The standard deviation for band-like infiltrate, spongiosis, itching, knee and elbow involvement, indicate that they have high variability of symptoms among patients.
- Minimum values are 0 for all clinical attributes, confirming absence of symptoms in some cases.
- Maximum values are 3 which are consistent with the defined clinical scoring system

```
df['class'].value_counts()

class
1    112
3     72
2     61
5     52
4     49
6     20
Name: count, dtype: int64

sns.countplot(x = 'class', data=df)
plt.title('Distribution of Skin Disease classes')
plt.show()
```



- The countplot reveals that the number of sampled varies significantly across different skin disease classes, indicating that the dataset is imbalanced.

```
sns.histplot(x= 'class', data=df)
plt.show()
```



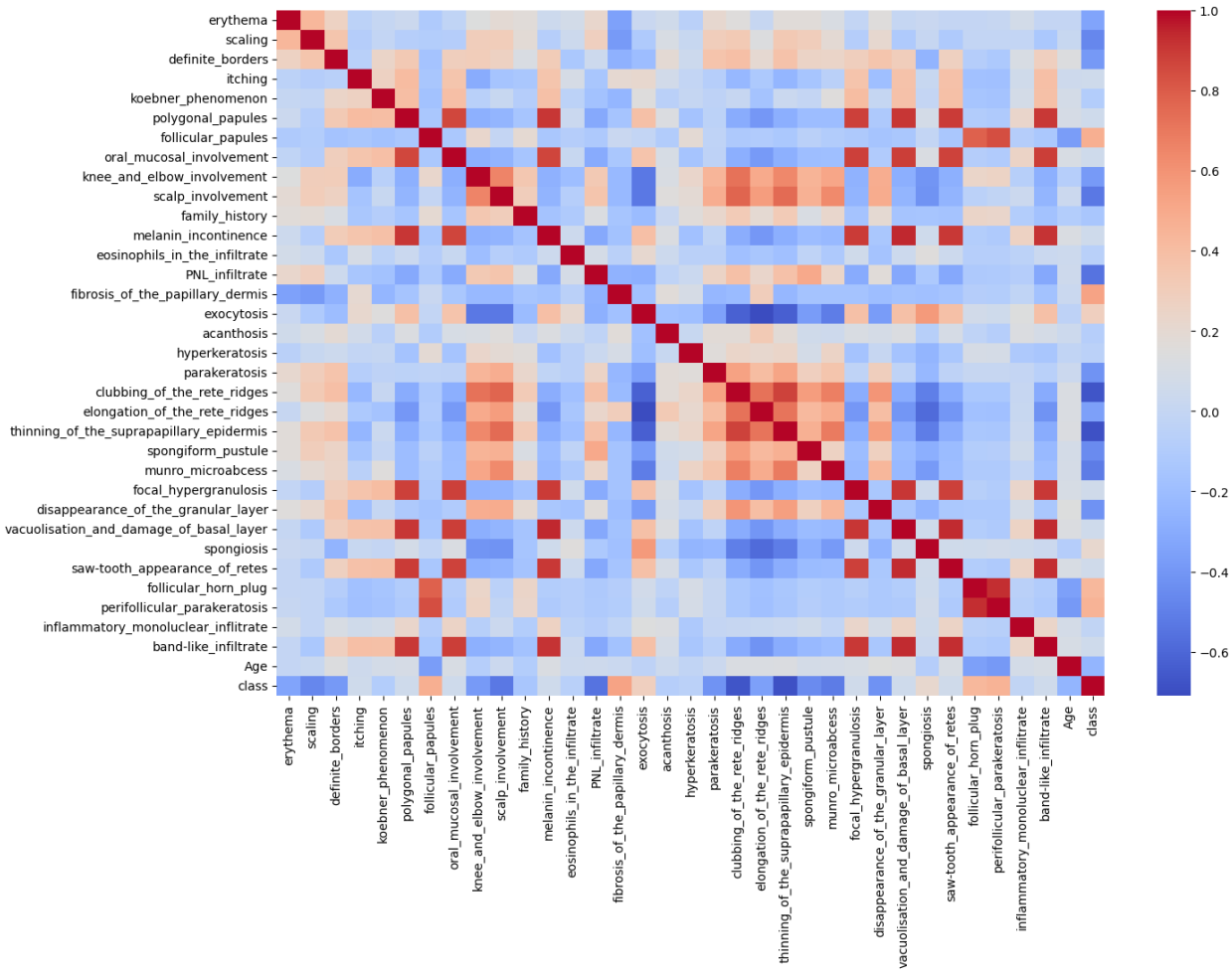

```
df.iloc[:,:-1].hist(figsize=(20,25))
plt.show()
```

erythema
scaling
definite_borders
itching
koebner_phenomenon
polygonal_papules
follicular_papules
oral_mucosal_involvement
knee_and_elbow_involvement
scalp_involvement
family_history
melanin_incontinence
eosinophils_in_the_infiltrate
PNL_infiltrate
fibrosis_of_the_papillary_dermis
exocytosis
acanthosis
hyperkeratosis
parakeratosis
clubbing_of_the_rete_ridges
elongation_of_the_rete_ridges
thinning_of_the_suprapapillary_epidermis
spongiform_pustule
munro_microabcess
focal_hypergranulosis
disappearance_of_the_granular_layer
vacuolisation_and_damage_of_basal_layer
spongiosis
saw-tooth_appearance_of_retes
follicular_horn_plug
perifollicular_parakeratosis
inflammatory_monoluclear_infiltrate
band-like_infiltrate
Age

OBSERVATION

- The histogram above shosw that most clinical features are concentrated at lower values (0 or 1), which indicates that many symptoms are absent or mild in a large number of patients.
- Histopathological features display a stronger skew toward zero, meaning that many microscopic abnomalities are not present in most cases.
- Histopathological attributes show greater variability, making hte useful for distinguishing between different skin diseases.

```
plt.figure(figsize=(15,10))
sns.heatmap(df.corr(), cmap='coolwarm')
plt.show()
```



- CORRELATION ANALYSIS
- The heatmap shows several strong positive correlations among related clinical features such as erythema, scaling, and definite bordes, indicating that these visible symptoms often occur together in inflammatory skin conditions.
- Strong correlations are also observed among histopathological features, particularly between acanthosis, hyperkeratosis, parakeratosis, and elongation of rete ridges.

- Features such as Munro microabscess and spongiform pustule show high correlation, reflecting their shared pathological origin in neutrophilic inflammation.
- The correlation between band like infiltrate and basal layer damage is medically relevant, as both are characteritic findings in lichen planus.
- Most clinical and histopathological features show weak correlation with each other, higlighting that microscopic findinds provide complementary information beyond visible symptoms.

Clinical Vs Histopathological Feature Insights

- Histopathological features are more discriminative than clinical features because they represent disease specific microscopic tissue changes, whereas clinical symptoms are often shared across mulitple skin conditions. The higher specificity and variability of histopathological findings enable better separation of disease classes, making them more effective for accurate classification.

Key Points

- Dataset is mostly clean
- Age required imputation
- No major calss imbalance
- Histopathological features are highly informative

Predictive Modeling (Machine Learning)

```
#Creating independent and dependent variable
X = df.drop('class',axis=1)
y = df['class']

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Creating training and testing data
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.3, random_state=42, stratify=y
)

y_test
```

```
182    1
246    1
322    4
```

```
313    3
218    4
      ..
197    2
364    3
329    1
335    1
178    3
Name: class, Length: 110, dtype: int64
```

LOGISTIC REGRESSION

- Logistic Regression is commonly used to estimate the probability that an instance belongs to a particular class. If the estimated probability is greater than 50% then the model predicts that the instance belongs to that class. It is a widely used statistical and machine learning algorithm for categorical target variable by estimating probabilities using the losgistic (sigmoid) function. Despite its simplicity, Logistic Regression performs effectively when the relationship between variables is approximately linear and serves as a strong baseline model. In medical classification tasks, it is valued for its interpretability, allowing clinicians to understand the influence of individual features on disease prediction.

```
# Model creation
from sklearn.linear_model import LogisticRegression
LR = LogisticRegression()
LR.fit(X_train,y_train)
```

```
LogisticRegression()
```

```
# Prediction
y_pred = LR.predict(X_test)
```

```
y_pred
```

```
array([1, 1, 4, 3, 4, 4, 6, 1, 3, 5, 4, 4, 3, 3, 2, 2, 3, 1, 3, 4, 2,
6,
       1, 4, 2, 1, 5, 3, 4, 1, 5, 2, 6, 3, 5, 1, 2, 6, 1, 1, 1, 5, 1,
1,
       4, 3, 5, 1, 3, 2, 5, 5, 1, 1, 4, 4, 3, 1, 3, 3, 1, 1, 4, 5, 3,
3,
       1, 1, 2, 1, 2, 3, 4, 4, 1, 2, 2, 1, 5, 3, 1, 2, 5, 5, 2, 1, 2,
1,
       1, 5, 3, 1, 2, 1, 4, 5, 4, 2, 1, 5, 1, 6, 3, 4, 6, 2, 3, 1, 1,
3])
```

```
y_pred_prob = LR.predict_proba(X_test)
```

```
y_pred_prob
```

```
array([[9.54368990e-01, 4.58917120e-03, 8.30230652e-04, 3.40924439e-
03,
```

```
        3.62408954e-02, 5.61468213e-04],
       [9.99953632e-01, 2.98968995e-06, 6.54009847e-06, 1.48391808e-
05,
        1.28159902e-05, 9.18315487e-06],
       [2.45205661e-03, 8.39411111e-02, 4.15037641e-03, 9.00851025e-
01,
        6.18853604e-03, 2.41689456e-03],
       [3.33484649e-04, 3.06334273e-05, 9.98657398e-01, 3.67996490e-
04,
        5.10294835e-04, 1.00192918e-04],
       [6.51801801e-04, 2.06844424e-02, 8.16191311e-04, 9.76926778e-
01,
        5.14639155e-04, 4.06147098e-04],
       [7.48272515e-04, 1.85284958e-03, 3.85588241e-04, 9.95402279e-
01,
        1.16341159e-03, 4.47599235e-04],
       [6.07840771e-04, 3.55494474e-03, 3.00661105e-04, 2.25888955e-
03,
        1.11312121e-03, 9.92164543e-01],
       [9.99826894e-01, 1.64208757e-06, 1.93133252e-05, 4.65521691e-
05,
        5.99423898e-05, 4.56557376e-05],
       [2.08262980e-04, 2.47403600e-03, 9.94606774e-01, 1.42397285e-
03,
        1.02844261e-03, 2.58511742e-04],
       [1.69023520e-03, 4.64352382e-02, 2.44627976e-03, 2.08562341e-
02,
        9.27293996e-01, 1.27801626e-03],
       [5.22575174e-04, 1.82160347e-03, 3.81714611e-04, 9.96340888e-
01,
        5.32295206e-04, 4.00923604e-04],
       [1.90098460e-03, 4.62304336e-01, 2.29008484e-03, 5.25154427e-
01,
        4.81678333e-03, 3.53338412e-03],
       [1.77935171e-04, 9.43451447e-06, 9.99446162e-01, 2.59508528e-
04,
        9.14006914e-05, 1.55587948e-05],
       [1.20531281e-04, 1.60820206e-05, 9.99683960e-01, 8.42412980e-
05,
        6.61454503e-05, 2.90401437e-05],
       [7.12753843e-04, 9.27590621e-01, 1.41500053e-03, 6.85851919e-
02,
        1.08598201e-03, 6.10450311e-04],
       [1.80806743e-02, 9.70712917e-01, 1.17245561e-03, 2.46621767e-
03,
        5.24469407e-03, 2.32304174e-03],
       [4.50378734e-05, 2.42990004e-03, 9.97163442e-01, 2.34664807e-
04,
        5.05405237e-05, 7.64148595e-05],
```

```
       [9.99569436e-01, 3.43400244e-05, 5.42596541e-05, 1.39486515e-
04,
        1.17591577e-04, 8.48861099e-05],
       [2.00120229e-03, 1.12317796e-03, 9.93387253e-01, 5.96665319e-
04,
        2.60031660e-03, 2.91384411e-04],
       [2.28627286e-03, 2.83379515e-02, 1.46759522e-03, 9.64019343e-
01,
        2.99567254e-03, 8.93164521e-04],
       [1.24368212e-03, 9.90387098e-01, 5.71106795e-04, 4.94850316e-
03,
        1.76358852e-03, 1.08602140e-03],
       [1.16636944e-02, 2.06255780e-03, 8.44180176e-04, 1.08421872e-
02,
        1.21332093e-02, 9.62454171e-01],
       [9.85279285e-01, 3.82559585e-06, 3.26776712e-04, 1.12616894e-
02,
        2.92539900e-03, 2.03024709e-04],
       [1.01489389e-03, 2.98008747e-03, 7.19246305e-04, 9.93869377e-
01,
        9.37028807e-04, 4.79366619e-04],
       [2.57541833e-03, 8.91270431e-01, 2.49758524e-03, 9.68166417e-
02,
        5.38722537e-03, 1.45269859e-03],
       [9.99798276e-01, 6.80879386e-06, 3.22293798e-05, 1.51335433e-
05,
        9.15731432e-05, 5.59787357e-05],
       [2.43656269e-03, 6.10632728e-04, 4.37486815e-04, 4.29115947e-
03,
        9.91777629e-01, 4.46529524e-04],
       [9.69457583e-06, 6.11108538e-06, 9.99862519e-01, 8.59265547e-
05,
        2.47534923e-05, 1.09956819e-05],
       [1.26777825e-03, 6.28107586e-03, 6.49763479e-04, 9.90313589e-
01,
        9.37420577e-04, 5.50373135e-04],
       [9.99945424e-01, 1.34719392e-06, 1.24877888e-05, 9.39821584e-
06,
        1.85038853e-05, 1.28393403e-05],
       [1.85579130e-04, 6.79176044e-04, 1.25361755e-04, 1.92526737e-
04,
        9.98770553e-01, 4.68029059e-05],
       [7.92498390e-02, 8.40614100e-01, 2.63907885e-03, 2.59400667e-
02,
        4.51744566e-02, 6.38245869e-03],
       [1.25369522e-03, 1.63255222e-04, 3.32888307e-04, 1.99440311e-
03,
        2.37648645e-03, 9.93879272e-01],
       [8.48252902e-04, 7.93821731e-04, 9.97488867e-01, 1.05964007e-
```

```
04,
        5.90848312e-04, 1.72246545e-04],
       [1.58640255e-02, 5.14226259e-02, 4.33533905e-03, 3.08539676e-
02,
        8.93670733e-01, 3.85330917e-03],
       [9.99924398e-01, 3.30147809e-05, 4.59121261e-06, 1.31502123e-
06,
        1.69299470e-05, 1.97510736e-05],
       [2.68681819e-04, 9.98219318e-01, 2.91738661e-04, 4.76705581e-
04,
        6.07230844e-04, 1.36324745e-04],
       [1.95138879e-01, 4.16067339e-02, 7.72599500e-03, 7.79902136e-
03,
        4.11473641e-02, 7.06582007e-01],
       [9.99989205e-01, 9.95531802e-08, 2.46790259e-06, 1.87113906e-
06,
        8.44365401e-07, 5.51209880e-06],
       [9.95544100e-01, 6.72899958e-05, 1.48243660e-04, 1.04277087e-
03,
        2.35104520e-03, 8.46550754e-04],
       [9.93312550e-01, 1.70028743e-05, 1.93009775e-04, 1.24251240e-
03,
        4.80354932e-03, 4.31375883e-04],
       [6.19564270e-04, 1.00492755e-03, 4.09131665e-04, 4.35718864e-
04,
        9.97230712e-01, 2.99945988e-04],
       [9.96380000e-01, 1.64028104e-05, 1.51956390e-04, 9.04973157e-
04,
        2.29319091e-03, 2.53476253e-04],
       [9.99862427e-01, 3.83119686e-07, 1.62669166e-05, 5.87740845e-
05,
        4.89700049e-05, 1.31786921e-05],
       [1.21915893e-02, 8.16310972e-02, 6.05256901e-03, 8.82192189e-
01,
        1.31510515e-02, 4.78150401e-03],
       [2.83288544e-03, 1.05590091e-04, 9.93655517e-01, 1.38579828e-
03,
        1.10739241e-03, 9.12816977e-04],
       [1.63268566e-04, 1.61243963e-04, 1.01037481e-04, 1.67142402e-
04,
        9.99373984e-01, 3.33237322e-05],
       [9.99342254e-01, 9.34996995e-06, 3.97186762e-05, 4.03091167e-
04,
        1.70244959e-04, 3.53408039e-05],
       [3.26984982e-04, 6.21478664e-05, 9.73218690e-01, 2.62642558e-
02,
        7.34844828e-05, 5.44369293e-05],
       [6.03098937e-05, 9.99364893e-01, 1.41861211e-04, 3.16490266e-
04,
```

```
        7.81604056e-05, 3.82855322e-05],
       [7.53092530e-03, 5.83549578e-04, 7.00730685e-04, 8.20908240e-
03,
        9.82495379e-01, 4.80332897e-04],
       [2.84035333e-04, 4.49451331e-05, 7.90226924e-05, 1.16170671e-
03,
        9.98359533e-01, 7.07566326e-05],
       [9.99508883e-01, 1.45439231e-05, 3.58698738e-05, 2.86063468e-
05,
        1.28914610e-04, 2.83181855e-04],
       [9.90853792e-01, 1.84265483e-03, 1.05313137e-03, 4.94035993e-
03,
        7.85015082e-04, 5.25046626e-04],
       [5.54185985e-04, 4.49633374e-02, 1.63065483e-03, 9.48259675e-
01,
        2.89339385e-03, 1.69875241e-03],
       [1.43983246e-02, 4.21087660e-04, 1.76159910e-03, 9.73767775e-
01,
        8.55525634e-03, 1.09595784e-03],
       [2.56971906e-05, 1.09950955e-05, 9.99899056e-01, 4.86062743e-
05,
        7.46734593e-06, 8.17839112e-06],
       [9.39188969e-01, 3.64791544e-02, 1.71297201e-03, 3.26755059e-
03,
        1.77650972e-02, 1.58625636e-03],
       [9.15773690e-05, 4.10625400e-06, 9.99277385e-01, 5.32364320e-
04,
        7.77238937e-05, 1.68432791e-05],
       [5.67229903e-05, 6.48705733e-04, 9.98752068e-01, 2.42856371e-
04,
        1.92090091e-04, 1.07556641e-04],
       [9.99335968e-01, 2.22819637e-05, 4.10904686e-05, 9.40310476e-
05,
        4.38063508e-04, 6.85647647e-05],
       [9.99820996e-01, 7.71450829e-08, 1.12351477e-05, 8.95739946e-
05,
        1.60614447e-05, 6.20564665e-05],
       [6.58707873e-04, 2.57821824e-03, 7.23055061e-04, 9.94800753e-
01,
        8.55796840e-04, 3.83468738e-04],
       [1.70730651e-04, 2.95500196e-04, 6.56610281e-05, 1.53659774e-
04,
        9.99147310e-01, 1.67138719e-04],
       [2.83219060e-03, 1.96316115e-04, 9.85217172e-01, 4.98073357e-
03,
        6.08882516e-03, 6.84762609e-04],
       [1.48244994e-03, 4.68377993e-03, 9.77745375e-01, 7.20326444e-
04,
        1.44657912e-02, 9.02277006e-04],
```

```
       [8.97135697e-01, 1.00321347e-01, 8.73928486e-04, 1.41808939e-
04,
        1.14594244e-03, 3.81276160e-04],
       [9.99930483e-01, 2.95028405e-07, 1.39673813e-05, 1.80462387e-
05,
        2.99956958e-05, 7.21256851e-06],
       [7.05328844e-04, 9.84308479e-01, 9.81948565e-04, 1.06929096e-
02,
        2.78755276e-03, 5.23780937e-04],
       [9.99911548e-01, 8.71159552e-06, 1.57602512e-05, 2.34619016e-
05,
        2.48024069e-05, 1.57159974e-05],
       [1.01061606e-02, 9.61263478e-01, 3.55950029e-03, 3.91739894e-
03,
        2.64718884e-03, 1.85062738e-02],
       [5.75295236e-05, 5.09506107e-05, 9.99030581e-01, 7.91177868e-
04,
        3.10055480e-05, 3.87556705e-05],
       [5.15640584e-04, 4.83969435e-04, 1.98068557e-04, 9.98418350e-
01,
        1.61661513e-04, 2.22309844e-04],
       [2.86186560e-02, 1.14783266e-03, 4.13001685e-01, 4.79133198e-
01,
        7.45617056e-02, 3.53692331e-03],
       [9.99841662e-01, 8.94853328e-06, 1.73232048e-05, 6.61309873e-
05,
        4.91637965e-05, 1.67713846e-05],
       [1.18833233e-04, 9.99751042e-01, 4.93211644e-05, 2.09246196e-
05,
        1.71307712e-05, 4.27482148e-05],
       [1.70227367e-02, 8.06564374e-01, 6.86613203e-03, 9.53898903e-
02,
        6.98603505e-02, 4.29651638e-03],
       [9.99942363e-01, 1.62058935e-05, 4.74181387e-06, 9.88972236e-
07,
        1.64583017e-05, 1.92419097e-05],
       [3.51661390e-03, 4.99928847e-03, 1.26942128e-03, 1.25793972e-
02,
        9.59882009e-01, 1.77532703e-02],
       [1.55530900e-05, 5.93279934e-04, 9.99300926e-01, 3.18079486e-
05,
        3.05182874e-05, 2.79148604e-05],
       [9.99130734e-01, 1.17644464e-05, 5.34617903e-05, 1.72151729e-
04,
        5.73104471e-04, 5.87831293e-05],
       [2.97558784e-02, 9.37170312e-01, 2.83849028e-03, 6.13345913e-
03,
        2.13111202e-02, 2.79074019e-03],
       [9.12823940e-04, 9.40589755e-04, 5.68649638e-04, 2.00439418e-
```

```
04,
        9.97308267e-01, 6.92304758e-05],
       [3.00375835e-03, 3.48666620e-03, 8.55076527e-04, 1.53451588e-
03,
        9.90494064e-01, 6.25918573e-04],
       [2.33182665e-04, 9.89925764e-01, 3.34190156e-04, 8.66628912e-
03,
        4.09371745e-04, 4.31202526e-04],
       [9.99499025e-01, 8.42288619e-07, 2.79085419e-05, 1.25191197e-
04,
        3.17669018e-04, 2.93637619e-05],
       [7.73818811e-03, 8.50112716e-01, 1.15730893e-02, 8.79341345e-
02,
        3.90605745e-02, 3.58129805e-03],
       [9.99790071e-01, 3.60537978e-05, 1.15165417e-05, 2.08082007e-
05,
        5.07411359e-05, 9.08095261e-05],
       [9.99940313e-01, 2.10296912e-06, 7.38119595e-06, 2.66188589e-
05,
        1.16623325e-05, 1.19213097e-05],
       [4.80455260e-04, 8.78562113e-04, 2.38159259e-04, 3.59268551e-
04,
        9.97837888e-01, 2.05666780e-04],
       [9.50905816e-05, 5.95136947e-06, 9.99776155e-01, 1.68092374e-
05,
        9.37042068e-05, 1.22898152e-05],
       [9.99768550e-01, 1.57591195e-05, 4.00334599e-05, 1.39230373e-
05,
        1.27412149e-04, 3.43220495e-05],
       [1.78558019e-02, 5.79181614e-01, 1.05732669e-02, 3.70853421e-
01,
        1.51507502e-02, 6.38514546e-03],
       [9.77983581e-01, 7.56806829e-03, 6.46043029e-04, 6.26089039e-
04,
        1.23738245e-02, 8.02393813e-04],
       [6.13322752e-04, 5.16726747e-02, 1.55192736e-03, 9.45039444e-
01,
        5.98777642e-04, 5.23853546e-04],
       [8.00282900e-05, 3.94223044e-05, 3.22031764e-05, 2.50860752e-
05,
        9.99777856e-01, 4.54041523e-05],
       [2.74467990e-03, 9.03155644e-02, 2.26887892e-03, 9.00934344e-
01,
        2.73621729e-03, 1.00031575e-03],
       [7.85943936e-04, 9.78035914e-01, 8.84298893e-04, 1.69395836e-
02,
        2.59024831e-03, 7.64011723e-04],
       [9.99947298e-01, 1.10775966e-05, 8.86454805e-06, 6.15611648e-
06,
```

```
        1.35613015e-05, 1.30428226e-05],
       [9.45931569e-04, 1.32903395e-04, 1.61114352e-04, 7.58130187e-
05,
        9.98081380e-01, 6.02857728e-04],
       [9.74994457e-01, 1.50692511e-02, 5.83963171e-04, 1.90684656e-
03,
        6.03196110e-03, 1.41352070e-03],
       [1.58848541e-05, 2.19033239e-06, 7.30269728e-06, 1.27804040e-
05,
        4.70494778e-05, 9.99914792e-01],
       [3.37785860e-04, 2.22935103e-05, 9.98141099e-01, 1.00371680e-
04,
        1.35576160e-03, 4.26882380e-05],
       [1.73895453e-02, 3.28872434e-01, 5.02649646e-03, 6.19193440e-
01,
        2.25774301e-02, 6.94065382e-03],
       [3.93448135e-04, 1.43955097e-02, 4.19737707e-04, 2.19919346e-
03,
        8.87909665e-04, 9.81704201e-01],
       [2.01408885e-02, 8.92409445e-01, 3.17387841e-03, 7.96205602e-
02,
        3.08282914e-03, 1.57239926e-03],
       [1.71075477e-04, 1.55898451e-05, 9.98484355e-01, 1.15573988e-
03,
        1.39296008e-04, 3.39434200e-05],
       [9.99584384e-01, 3.86377410e-07, 2.57516532e-05, 2.67754697e-
04,
        1.08538608e-04, 1.31848710e-05],
       [9.99432054e-01, 4.73526438e-06, 4.51765383e-05, 1.70841192e-
04,
        3.01102252e-04, 4.60909981e-05],
       [5.44668085e-05, 1.17753681e-03, 9.98438402e-01, 1.36794447e-
04,
        1.23035825e-04, 6.97640277e-05]])
```

```
df['class'].value_counts()
```

```
class
1    112
3     72
2     61
5     52
4     49
6     20
Name: count, dtype: int64
```

```
from sklearn.metrics import confusion_matrix, accuracy_score,
recall_score, precision_score, classification_report, f1_score
```

```
cm = confusion_matrix(y_test, y_pred)
print (cm)
```

```
[[34  0  0  0  0  0]
 [ 0 16  0  2  0  0]
 [ 0  0 21  1  0  0]
 [ 0  1  0 14  0  0]
 [ 0  0  0  0 15  0]
 [ 0  0  0  0  0  6]]
```

```
recall=recall_score(y_test, y_pred, average='weighted')
recall
```

```
0.9636363636363636
```

```
print (classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           1       1.00      1.00      1.00        34
           2       0.94      0.89      0.91        18
           3       1.00      0.95      0.98        22
           4       0.82      0.93      0.88        15
           5       1.00      1.00      1.00        15
           6       1.00      1.00      1.00         6

    accuracy                           0.96       110
   macro avg       0.96      0.96      0.96       110
weighted avg       0.97      0.96      0.96       110
```

```
precision=precision_score(y_test, y_pred, average='weighted')
precision
```

```
0.9663101604278074
```

```
f1score=f1_score(y_test, y_pred, average='weighted')
f1score
```

```
0.9642774086378738
```

```
cr=classification_report(y_test, y_pred)
print(cr)
```

```
              precision    recall  f1-score   support

           1       1.00      1.00      1.00        34
           2       0.94      0.89      0.91        18
           3       1.00      0.95      0.98        22
           4       0.82      0.93      0.88        15
           5       1.00      1.00      1.00        15
           6       1.00      1.00      1.00         6
```

```
    accuracy                           0.96       110
   macro avg       0.96      0.96      0.96       110
weighted avg       0.97      0.96      0.96       110
```

- OBSERVATION
- Logistic Regression was used as a baseline classification model for predicting skin disease classes.
- The model performed reasonably well on linearly separable classes, showing stable and consistent perdictions.
- Feature scaling using StandardScaler significantly improved the model's performance.
- The classificaiton report showed imbalanced precision and recall for certain classes, especially those with fewer samples.

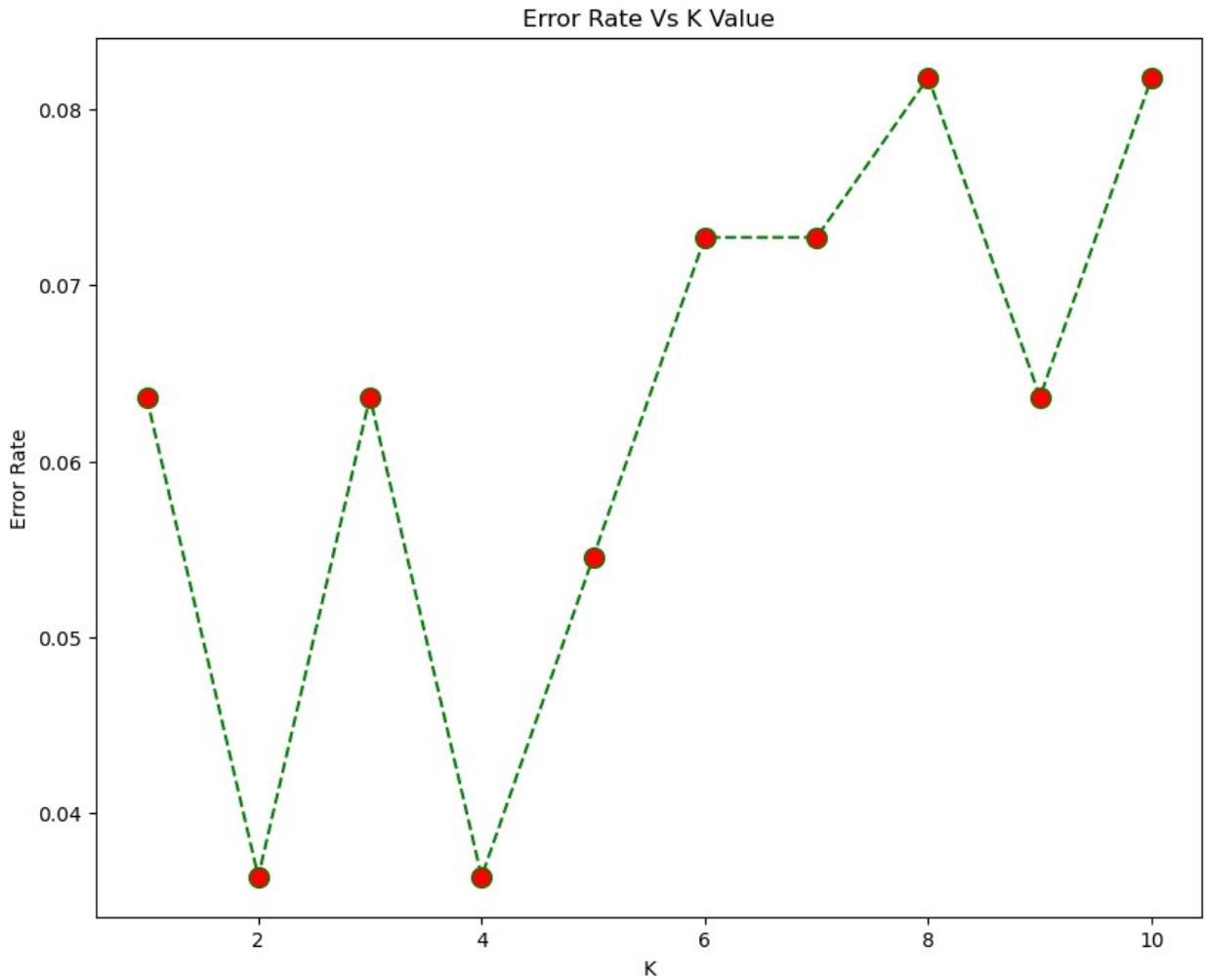
KNN

- K-Nearest Neighbors (KNN) is a type of supervised learning algorithm which is used for both regression and classification purposes, but mostly it is used for the latter. Given the dataset, with different classes, KNN tries to predict the correct class of test data by calculating the distance between the test data and all the training points. It then selects the k points which are closest to the test data. The closeness beween data points is typically measured using distance metrics such as Euclidean distance. Once the points are selected, the algorithm calculates the probability (in case of classification) of the test point belonging to the classes of the k training points and the regression problem, the predicted value is the mean of the k selected training points. In medical diagnosis problems, including skin disease classification, KNN is particularly useful because it relies on similarity patterns among patient records. When combined with proper feature scaling and balanced data, KNN can effectively capture complex relationships between clinical histopathological features, making it a valuable baseline model for comparison.

```
# Importing libraries
from sklearn.neighbors import KNeighborsClassifier
from sklearn.neighbors import KNeighborsRegressor
import warnings
warnings.filterwarnings('ignore')

# taking optimal k to determine how many nearest neighbors to create
# creating a list to store the error values for each k
error_rate=[]

for i in range(1,11):
    knn = KNeighborsClassifier(n_neighbors=i)
    knn.fit(X_train,y_train)
    pred_i = knn.predict(X_test)
    error_rate.append(np.mean(pred_i !=y_test))

error_rate
```

```
[np.float64(0.06363636363636363),
 np.float64(0.03636363636363636),
 np.float64(0.06363636363636363),
 np.float64(0.03636363636363636),
 np.float64(0.05454545454545454),
 np.float64(0.07272727272727272),
 np.float64(0.07272727272727272),
 np.float64(0.08181818181818182),
 np.float64(0.06363636363636363),
 np.float64(0.08181818181818182)]
```

```
# Plotting the k-value and error rate

plt.figure(figsize=(10,8))
plt.plot(range(1,11), error_rate, color='green', linestyle = 'dashed',
        marker = 'o', markerfacecolor = 'red', markersize=10)
plt.title('Error Rate Vs K Value')
plt.xlabel('K')
plt.ylabel('Error Rate')
plt.show()
```




```
from sklearn.metrics import mean_squared_error

rmse_val = [] #creating list to store rmse values for different k
for K in range(1,25):
    model = KNeighborsRegressor(n_neighbors = K)

    model.fit(X_train, y_train) #to fit the model
    pred = model.predict(X_test) #to make prediction on test set
    error = np.sqrt(mean_squared_error(y_test, pred)) #calculate rmse
    rmse_val.append(error) #store the rmse values
    print ('RMSE Value for k=', 'is:', error)
```

```
RMSE Value for k= is: 0.4472135954999579
RMSE Value for k= is: 0.40451991747794525
RMSE Value for k= is: 0.4168181542799855
RMSE Value for k= is: 0.3981491269269936
RMSE Value for k= is: 0.44680685271867043
RMSE Value for k= is: 0.4561587531559688
RMSE Value for k= is: 0.4590867760059366
```

```
RMSE Value for k= is: 0.44498467797932506
RMSE Value for k= is: 0.43474259319102343
RMSE Value for k= is: 0.4412791324403101
RMSE Value for k= is: 0.43486312258793236
RMSE Value for k= is: 0.42684536977337556
RMSE Value for k= is: 0.42747239906621015
RMSE Value for k= is: 0.4382034498390387
RMSE Value for k= is: 0.447213595499958
RMSE Value for k= is: 0.4431453947736464
RMSE Value for k= is: 0.4646348013057787
RMSE Value for k= is: 0.46850886989173196
RMSE Value for k= is: 0.48250688438374995
RMSE Value for k= is: 0.4889459906964999
RMSE Value for k= is: 0.49827573232253897
RMSE Value for k= is: 0.5143142604744915
RMSE Value for k= is: 0.5301528441410882
RMSE Value for k= is: 0.5420598805864664
```

```
# Fitting the data into KNN model to know how well it performs:
knn1 = KNeighborsClassifier(n_neighbors=5)
knn1.fit(X_train, y_train)
```

```
KNeighborsClassifier()
```

```
y_pred1 = knn1.predict(X_test) #predict
```

```
# Checking the accuracy score
print ('The Accuracy Score is:', accuracy_score(y_test, y_pred1))
```

```
The Accuracy Score is: 0.9454545454545454
```

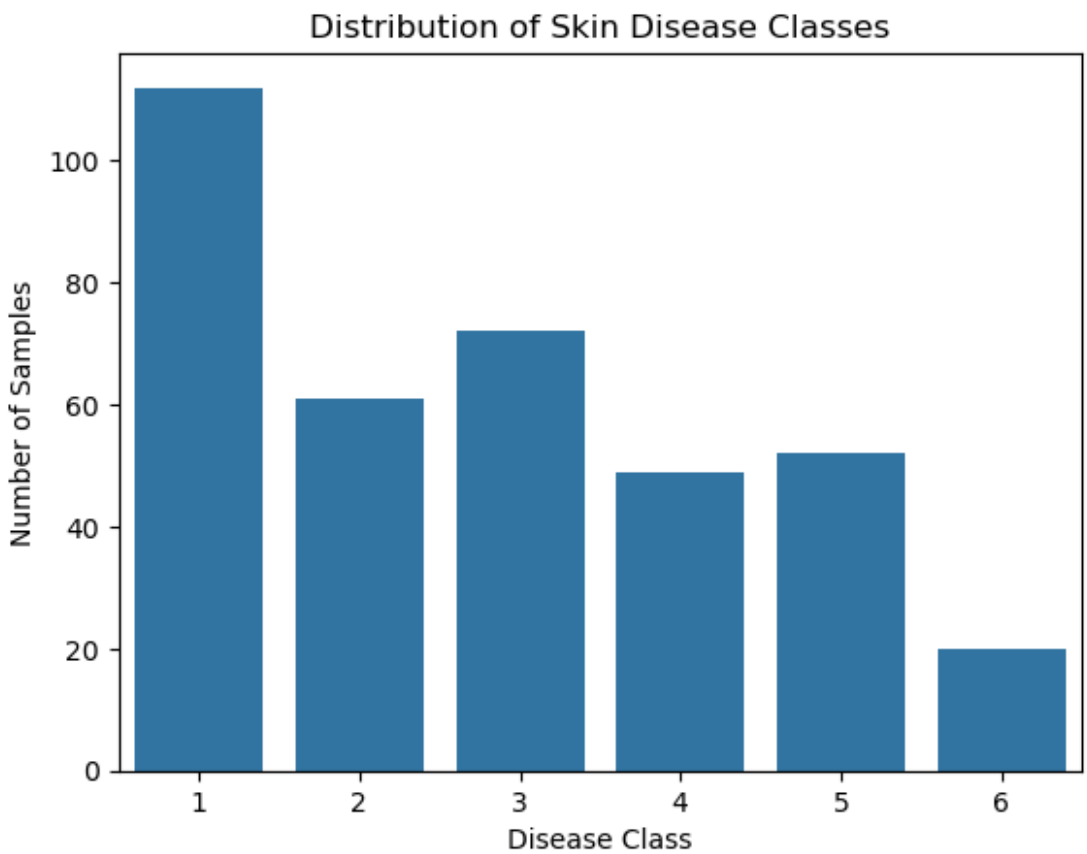
```
print (classification_report(y_test, y_pred1))
```

```
              precision    recall  f1-score   support

           1       1.00      0.97      0.99        34
           2       0.93      0.78      0.85        18
           3       1.00      0.95      0.98        22
           4       0.75      1.00      0.86        15
           5       1.00      1.00      1.00        15
           6       1.00      1.00      1.00         6

    accuracy                           0.95       110
   macro avg       0.95      0.95      0.94       110
weighted avg       0.95      0.95      0.95       110
```

```
recall1 = recall_score(y_test, y_pred1, average='weighted')
recall1
```

```
0.9454545454545454
```

```
sns.countplot(x = 'class', data=df)
plt.title('Distribution of Skin Disease Classes')
plt.xlabel('Disease Class')
plt.ylabel('Number of Samples')
plt.show() #imbalanced data
```



- Imbalanced data typically refers to a classification problem where the number of observations per class is not equally distributed.
- Synthetic Minority Over-sampling Technique (SMOTE) is a technique that generates new observations by interpolating between observations in the original dataset. Interpolation is done with the help of KNN algorithm.

```
!pip install imblearn

Requirement already satisfied: imblearn in c:\users\jamir\onedrive\
attachments\lib\site-packages (0.0)
Requirement already satisfied: imbalanced-learn in c:\users\jamir\
onedrive\attachments\lib\site-packages (from imblearn) (0.14.1)
Requirement already satisfied: numpy<3,>=1.25.2 in c:\users\jamir\
onedrive\attachments\lib\site-packages (from imbalanced-learn-
>imblearn) (2.1.3)
Requirement already satisfied: scipy<2,>=1.11.4 in c:\users\jamir\
onedrive\attachments\lib\site-packages (from imbalanced-learn-
```

```
>imblearn) (1.15.3)
Requirement already satisfied: scikit-learn<2,>=1.4.2 in c:\users\
jamir\onedrive\attachments\lib\site-packages (from imbalanced-learn-
>imblearn) (1.6.1)
Requirement already satisfied: sklearn-compat<0.2,>=0.1.5 in c:\users\
jamir\onedrive\attachments\lib\site-packages (from imbalanced-learn-
>imblearn) (0.1.5)
Requirement already satisfied: joblib<2,>=1.2.0 in c:\users\jamir\
onedrive\attachments\lib\site-packages (from imbalanced-learn-
>imblearn) (1.4.2)
Requirement already satisfied: threadpoolctl<4,>=2.0.0 in c:\users\
jamir\onedrive\attachments\lib\site-packages (from imbalanced-learn-
>imblearn) (3.5.0)
```

```
!pip install -U imbalanced-learn
```

```
Requirement already satisfied: imbalanced-learn in c:\users\jamir\
onedrive\attachments\lib\site-packages (0.14.1)
Requirement already satisfied: numpy<3,>=1.25.2 in c:\users\jamir\
onedrive\attachments\lib\site-packages (from imbalanced-learn) (2.1.3)
Requirement already satisfied: scipy<2,>=1.11.4 in c:\users\jamir\
onedrive\attachments\lib\site-packages (from imbalanced-learn)
(1.15.3)
Requirement already satisfied: scikit-learn<2,>=1.4.2 in c:\users\
jamir\onedrive\attachments\lib\site-packages (from imbalanced-learn)
(1.6.1)
Requirement already satisfied: sklearn-compat<0.2,>=0.1.5 in c:\users\
jamir\onedrive\attachments\lib\site-packages (from imbalanced-learn)
(0.1.5)
Requirement already satisfied: joblib<2,>=1.2.0 in c:\users\jamir\
onedrive\attachments\lib\site-packages (from imbalanced-learn) (1.4.2)
Requirement already satisfied: threadpoolctl<4,>=2.0.0 in c:\users\
jamir\onedrive\attachments\lib\site-packages (from imbalanced-learn)
(3.5.0)
```

```
# Applying SMOTE to balance the data
from imblearn.over_sampling import SMOTE

smote = SMOTE()

y_train.value_counts()
```

```
class
1    78
3    50
2    43
5    37
4    34
6    14
Name: count, dtype: int64
```

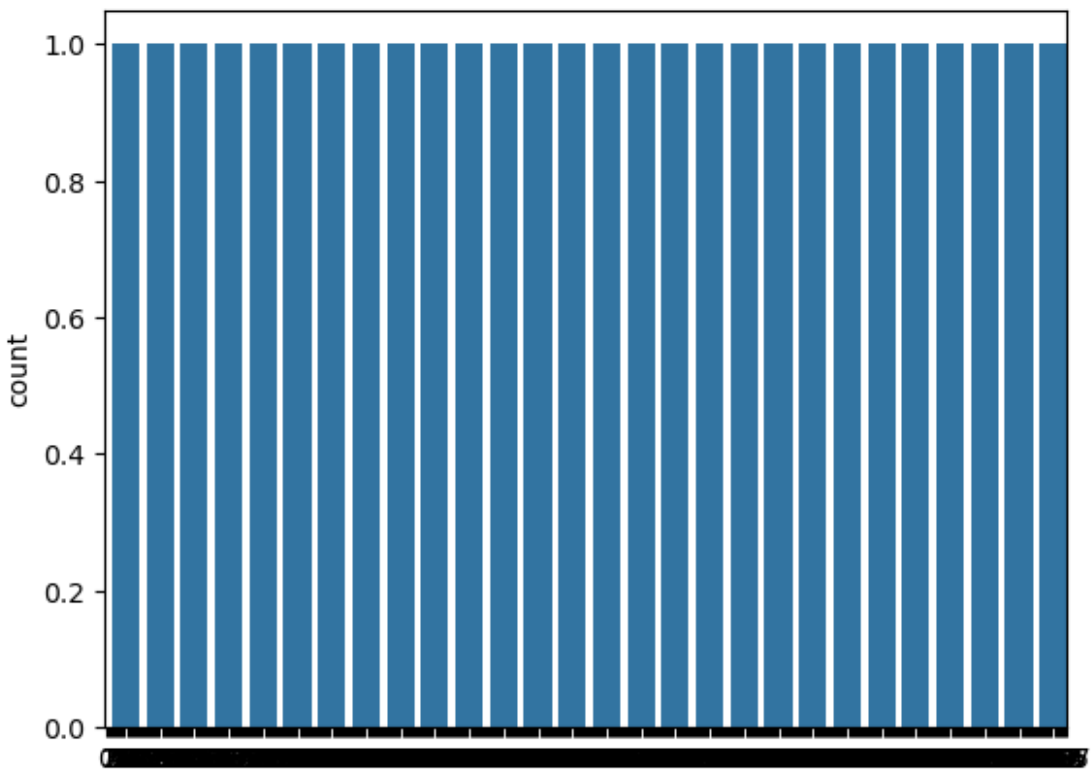
```
X_train_smote, y_train_smote =
smote.fit_resample(X_train.astype('float'),
                                                y_train)

y = np.array(y_train_smote)

y
```

```
array([1, 1, 4, 4, 4, 4, 1, 1, 1, 4, 3, 1, 3, 4, 2, 3, 5, 3, 2, 5, 1,
4,
       1, 5, 2, 5, 1, 5, 4, 1, 4, 1, 1, 4, 1, 2, 1, 3, 3, 6, 5, 1, 2,
4,
       1, 1, 1, 4, 1, 6, 5, 1, 1, 1, 2, 4, 5, 1, 1, 2, 2, 4, 1, 5, 5,
4,
       3, 3, 1, 2, 3, 3, 1, 2, 1, 1, 2, 5, 6, 2, 5, 4, 1, 3, 3, 1, 5,
1,
       2, 2, 2, 5, 3, 1, 2, 6, 2, 1, 4, 3, 3, 3, 3, 5, 2, 4, 3, 5, 3,
1,
       4, 4, 5, 1, 2, 5, 2, 3, 5, 3, 5, 3, 5, 5, 3, 2, 6, 2, 6, 3, 1,
4,
       1, 2, 1, 1, 1, 3, 5, 2, 1, 2, 3, 2, 1, 1, 3, 3, 3, 3, 5, 5, 1,
1,
       2, 2, 4, 5, 1, 4, 2, 3, 3, 3, 2, 3, 6, 5, 5, 1, 3, 1, 5, 1, 2,
2,
       2, 1, 2, 1, 2, 5, 1, 5, 2, 6, 1, 1, 5, 4, 1, 4, 5, 6, 3, 3, 4,
1,
       4, 6, 1, 6, 4, 6, 1, 2, 2, 3, 2, 4, 3, 3, 1, 1, 1, 4, 1, 1, 3,
1,
       1, 3, 3, 1, 2, 1, 3, 5, 1, 4, 1, 3, 3, 6, 1, 5, 3, 6, 3, 3, 4,
3,
       2, 4, 1, 2, 1, 4, 2, 1, 1, 1, 5, 5, 1, 1, 2, 2, 2, 2, 2, 2, 2,
2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
2,
       2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3,
3,
       3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4,
4,
       4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4,
4,
       4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5,
5,
       5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5,
5,
       5, 5, 5, 5, 5, 5, 5, 5, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6,
6,
       6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6,
6,
       6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6,
```

```
6,
       6, 6, 6, 6, 6, 6])
```

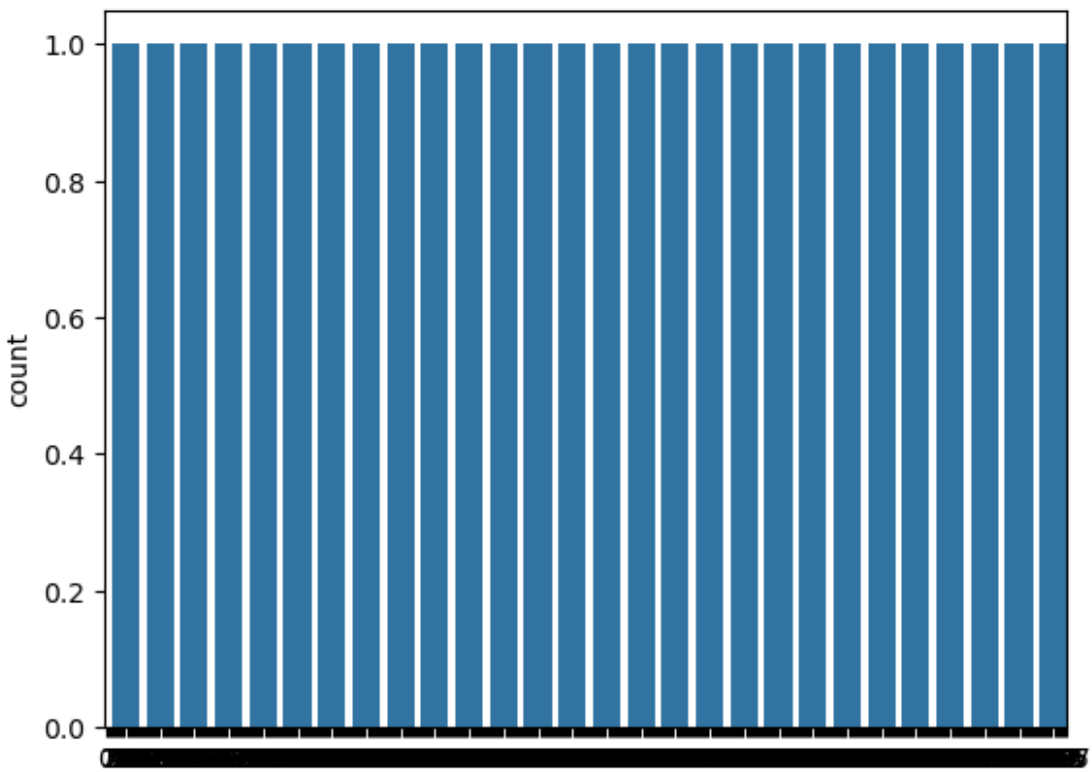
```
sns.countplot(y_train_smote)
```

```
<Axes: ylabel='count'>
```




```
sns.countplot(y)
```

```
<Axes: ylabel='count'>
```



- The countplot of the target variable after applying SMOTE shows a uniform distribution across all skin disease classes. This indicates that class imbalance present in the original training data has been effectively addressed. By balancing the classes, the model is less biased toward majority classes and is able to learn representative patterns-from all disease categories, leading to improved predictive performance.

```
from collections import Counter
print ('Actual Classes', Counter(y_train))
print ('SMOTE Classes', Counter(y_train_smote))
# the counter class is used to examine the distribution of target
classes before and after applying SMOTE
```

```
Actual Classes Counter({1: 78, 3: 50, 2: 43, 5: 37, 4: 34, 6: 14})
SMOTE Classes Counter({1: 78, 4: 78, 3: 78, 2: 78, 5: 78, 6: 78})
```

```
knn2 = KNeighborsClassifier(n_neighbors=5)
knn2.fit(X_train_smote, y_train_smote)
```

```
KNeighborsClassifier()
```

```
y_pred2 = knn2.predict(X_test)
```

```
from sklearn.metrics import f1_score
f1score1 = f1_score(y_test, y_pred2, average ='weighted')
```

```
print ('The Accuracy Score is:', accuracy_score(y_test, y_pred2))
print ('The f1_score is:', f1score1)
```

```
The Accuracy Score is: 0.9272727272727272
The f1_score is: 0.9286424425746688
```

```
print (classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

           1       1.00      1.00      1.00        34
           2       0.94      0.89      0.91        18
           3       1.00      0.95      0.98        22
           4       0.82      0.93      0.88        15
           5       1.00      1.00      1.00        15
           6       1.00      1.00      1.00         6

    accuracy                           0.96       110
   macro avg       0.96      0.96      0.96       110
weighted avg       0.97      0.96      0.96       110
```

- OBSERVATION
- KNN achieved good classification performance for the skin disease dataset, particularly for classes with distinct clinical and histopathological features.
- KNN showed misclassifications among skin diseases with overlapping symptoms, reflecting the inherent similarities between certain dermatological conditions.
- KNN predictions were intuitive, as classifications were based on the majority class of nearest clinical cases.
- This makes the model conceptually understandable, though it lacks explicit feature importance.

SVM

- SVM is the Supervised Machine Learning algorithm used for both classification, regression. Given the dataset, the algorithm tries to divide the data using hyperplanes and then makes the predictions. SVM is a non-probabilistic linear classifier. SVM constructs a best line or the decision boundary called Hyperplane which can be used for classification or regression or outlier detection. This hyperplane creates 2 margin lines parallel to it which have some distance so that it can distinctly classify the data points. The distance between the 2 margin lines are called marginal distance. These 2 margin lines passes through the most nearest positive points and the most nearest negative points. Those points through which the margin lines pass are called support vectors. Support vectors are important as it helps to determine the maximum distance of the marginal plane.

```
from sklearn.svm import SVC

svm_model = SVC(kernel='rbf', C=1.0, gamma='scale')
svm_model.fit(X_train_smote, y_train_smote)
```

```
SVC()

y_pred3 = svm_model.predict(X_test)

confusion_matrix(y_test, y_pred3)
```

```
array([[34,  0,  0,  0,  0,  0],
       [ 0, 15,  0,  3,  0,  0],
       [ 1,  0, 21,  0,  0,  0],
       [ 0,  2,  0, 13,  0,  0],
       [ 0,  0,  0,  0, 15,  0],
       [ 0,  0,  0,  0,  0,  6]])
```

```
from sklearn.metrics import f1_score
f1_score1 = f1_score(y_test, y_pred3, average='weighted')

print ('Accuracy:', accuracy_score(y_test, y_pred3))
print ('f1_score:', f1_score1)
print (classification_report(y_test, y_pred3))
```

```
Accuracy: 0.9454545454545454
f1_score: 0.9454985005426964
              precision    recall  f1-score   support

           1       0.97      1.00      0.99        34
           2       0.88      0.83      0.86        18
           3       1.00      0.95      0.98        22
           4       0.81      0.87      0.84        15
           5       1.00      1.00      1.00        15
           6       1.00      1.00      1.00         6

    accuracy                           0.95       110
   macro avg       0.94      0.94      0.94       110
weighted avg       0.95      0.95      0.95       110
```

- OBSERVATION
- The SVM model achieved high classification accuracy compared to linear models.
- Using the RBF kernel, SVM effectively captured non-linear relationships between clinical attributes.
- Model interpretability is limited as SVM does not provide direct faeture importance.

Decision Tree

- A decision tree is a supervised machine learning algorithm used for classification and regression tasks. It works by recursively splitting the dataset into subsets based on feature values, forming a tree-like structure of decision rules. Each internal mode represents a test on a feature, each branch corresponds to an outcome of the test, and

each leaf node represents a final prediction. Decision trees are widely used due to their simplicity,ability to handle both numerical and categorical data and use of interpretation.

```
from sklearn.tree import DecisionTreeClassifier

dt = DecisionTreeClassifier()
dt.fit(X_train_smote, y_train_smote)
y_predt = dt.predict(X_test)
y_predt
```

```
array([1, 1, 4, 3, 4, 4, 6, 1, 3, 5, 4, 4, 3, 3, 2, 4, 3, 1, 3, 4, 2,
6,
       1, 4, 2, 1, 5, 3, 4, 1, 5, 1, 6, 3, 5, 1, 2, 6, 1, 1, 1, 5, 1,
1,
       4, 3, 5, 1, 3, 2, 5, 5, 1, 1, 4, 1, 3, 1, 3, 3, 1, 1, 4, 5, 1,
3,
       1, 1, 2, 1, 6, 3, 4, 5, 1, 2, 4, 1, 5, 3, 1, 1, 5, 5, 2, 1, 2,
1,
       1, 5, 3, 1, 4, 1, 4, 5, 4, 2, 1, 5, 1, 6, 3, 2, 6, 2, 3, 1, 1,
3])
```

```
y_train_predict = dt.predict(X_train_smote)
```

```
y_train_predict
```

```
array([1, 1, 4, 4, 4, 4, 1, 1, 1, 4, 3, 1, 3, 4, 2, 3, 5, 3, 2, 5, 1,
4,
       1, 5, 2, 5, 1, 5, 4, 1, 4, 1, 1, 4, 1, 2, 1, 3, 3, 6, 5, 1, 2,
4,
       1, 1, 1, 4, 1, 6, 5, 1, 1, 1, 2, 4, 5, 1, 1, 2, 2, 4, 1, 5, 5,
4,
       3, 3, 1, 2, 3, 3, 1, 2, 1, 1, 2, 5, 6, 2, 5, 4, 1, 3, 3, 1, 5,
1,
       2, 2, 2, 5, 3, 1, 2, 6, 2, 1, 4, 3, 3, 3, 3, 5, 2, 4, 3, 5, 3,
1,
       4, 4, 5, 1, 2, 5, 2, 3, 5, 3, 5, 3, 5, 5, 3, 2, 6, 2, 6, 3, 1,
4,
       1, 2, 1, 1, 1, 3, 5, 2, 1, 2, 3, 2, 1, 1, 3, 3, 3, 3, 5, 5, 1,
1,
       2, 2, 4, 5, 1, 4, 2, 3, 3, 3, 2, 3, 6, 5, 5, 1, 3, 1, 5, 1, 2,
2,
       2, 1, 2, 1, 2, 5, 1, 5, 2, 6, 1, 1, 5, 4, 1, 4, 5, 6, 3, 3, 4,
1,
       4, 6, 1, 6, 4, 6, 1, 2, 2, 3, 2, 4, 3, 3, 1, 1, 1, 4, 1, 1, 3,
1,
       1, 3, 3, 1, 2, 1, 3, 5, 1, 4, 1, 3, 3, 6, 1, 5, 3, 6, 3, 3, 4,
3,
       2, 4, 1, 2, 1, 4, 2, 1, 1, 1, 5, 5, 1, 1, 2, 2, 2, 2, 2, 2, 2,
2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
2,
```

```
       2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3,
3,
       3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4,
4,
       4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4,
4,
       4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5,
5,
       5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 5,
5,
       5, 5, 5, 5, 5, 5, 5, 5, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6,
6,
       6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6,
6,
       6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6,
6,
       6, 6, 6, 6, 6, 6])
```

```
acc_train = accuracy_score(y_train_smote, y_train_predict)
acc_train
#overfit model
```

```
1.0
```

```
print (classification_report(y_train_smote, y_train_predict))
```

```
              precision    recall  f1-score   support

           1       1.00      1.00      1.00        78
           2       1.00      1.00      1.00        78
           3       1.00      1.00      1.00        78
           4       1.00      1.00      1.00        78
           5       1.00      1.00      1.00        78
           6       1.00      1.00      1.00        78

    accuracy                           1.00       468
   macro avg       1.00      1.00      1.00       468
weighted avg       1.00      1.00      1.00       468
```

```
test_acc = accuracy_score(y_test, y_predt)
test_acc #testing accuracy
```

```
0.9181818181818182
```

```
test_f1 = f1_score(y_test, y_predt, average='weighted')
test_f1
```

```
0.9149371148564696
```

```
print (classification_report(y_test, y_predt))
```

```
              precision    recall  f1-score   support

           1       0.89      1.00      0.94        34
           2       1.00      0.67      0.80        18
           3       1.00      0.91      0.95        22
           4       0.82      0.93      0.88        15
           5       0.94      1.00      0.97        15
           6       0.86      1.00      0.92         6

    accuracy                           0.92       110
   macro avg       0.92      0.92      0.91       110
weighted avg       0.93      0.92      0.91       110
```

```
pd.crosstab(y_test, y_predt)
```

```
col_0   1   2   3   4   5  6
class
1      34   0   0   0   0  0
2       2  12   0   3   0  1
3       1   0  20   0   1  0
4       1   0   0  14   0  0
5       0   0   0   0  15  0
6       0   0   0   0   0  6
```

```
from sklearn.model_selection import cross_val_score

scores_after =
cross_val_score(dt,X_train,y_train,cv=3,scoring='f1_weighted')
print (scores_after)

print ('Cross validation score:', scores_after.mean())
print ('Std:', scores_after.std())
```

```
[0.94088679 0.92943636 0.92851036]
Cross validation score: 0.9329445057723857
Std: 0.005628753411467352
```

```
#Stratified K-fold CV
from sklearn.model_selection import StratifiedKFold
skf = StratifiedKFold(n_splits=5)
scores_after =
cross_val_score(dt,X_train,y_train,cv=skf,scoring='f1_weighted')
print (scores_after)

print ('Cross validation score:', scores_after.mean())
print ('Std:', scores_after.std())
```

```
[0.90118378 0.92018136 0.98001689 0.93976392 0.96078431]
Cross validation score: 0.9403860523669036
Std: 0.02804327670975381
```

- OBSERVATION
- Cross validation revealed that the decision tree model showed noticeable variation in performance across different folds, indicating sensitivity to training data splits.
- The average cross validition accuracy was lower than the training accuracy, confirming the presence of overfitting.
- Cross validation helped provide a more reliable estimate of generalisation performance comapred to a single train-test split.

Random Forest

- Random Forest is an ensemble machine learning algorithm widely used for classification and regression tasks. It operates by constructing multiple decision tree during training and combining their prediction throuhh majority voting to produce a final output. By introducing randomness in both data sampling and feature selection, random forest reduces overfitting and improves generalization compared to a single decision tree. Its ability to handle high dimensional data, capture complex non-linear relationships, and provide robust performance makes it particularly suitable for medical applications such as skin disease classification.

```
from sklearn.ensemble import RandomForestClassifier

rf_clf = RandomForestClassifier(n_estimators=100) #object creation,
taking 100 decision tree in random forest
rf_clf.fit(X_train_smote, y_train_smote)

RandomForestClassifier()

y_predict = rf_clf.predict(X_test)

y_predt1 = rf_clf.predict(X_train_smote)

F1_score1 = f1_score(y_test, y_predict, average = 'weighted')

# Testing score
print ('Accuracy:', accuracy_score(y_test, y_predict))
print ('F1_score:', F1_score1)

Accuracy: 0.9636363636363636
F1_score: 0.9633563779673835

# Training score
print ('Accuracy:', accuracy_score(y_train_smote, y_predt1))
print ('F1_score:', f1_score(y_train_smote, y_predt1,
average='weighted'))

Accuracy: 1.0
F1_score: 1.0
```

Comparison of model performance

- Multiple machine learning models were trained and evaluated for the multiclass skin disease classification task. Model performance was measured using the weighted F1-score, which balances precision and recall while accounting for class imbalance.
- In this study, the weighted F1 score was used as the primary and sole evaluation metric. Given the multiclass and imbalanced nature of the skin disease dataset, the F1 score provides a balanced measure of precision and recall, which is particularly important in medical diagnosis tasks. The weighted averaging method further accounts for class distribution, making it a reliable indicator of overall comparison and selection.
- Logistic Regression achieved th ehighest weighted F1 score, indicating superior overall classification performance. Support Vector Machine(SVM) and Random Forest produced comparable results, showing strong predictive capability but with increased model complexity. K-Nearest (KNN) achieved a slightly lower F1 score and was sensitive to data distribution and parameter selection. The Decision Tree model recored the lowest F1 score suggesting overfitting and reduced generalisation performance.

```
print ('The following are the F1 score for different models:-')
print ('Logistic Regression-', f1score )
print ('KNN-',f1score1)
print ('SVM-',f1_score1)
print ('Decision Tree-',test_f1)
print ('Random Forest-',F1_score1)
```

```
The following are the F1 score for different models:-
Logistic Regression- 0.9642774086378738
KNN- 0.9286424425746688
SVM- 0.9454985005426964
Decision Tree- 0.9149371148564696
Random Forest- 0.9633563779673835
```

Best performing Model Selection

- Based on the weighted F1 score evaluation, Logistic Regression is selected as the best performing model for this project. It achieved th ehighest F1 score among all models and demonstrated consistent and reliable performance across classes
- In addition to achieving the highest weighted F1 score, the model offers high interpretability, allowing clinicains to understand the influence of individual features on disease prediction. This interpretability is especially important in medical applications, where transparency and trust are critical. Furthermore, Logistic Regression is computationally efficient, less prone to overfitting amd easier to deploy compared to more complex models such as SVM and Random Forest.

Suggestions to Doctors for Early Diagnosis of Skin Diseases

Important Features Identified by the Model

- The machine learning models identified several clinical and histopathological features as key contributors to accurate skin disease classification. Features such as erythema, scaling, definite borders, itching, knee and elbow involvment, scalp involvment, and family history were consistently important. Histopathological indicators including acanthosis, parakeratosis, inflammatory inifltrate, and elongation of rete ridges also played a significant role in differentiating similar skin conditions

Clinical and Histopathological Relevance

- Many erythemato-squamous diseases share common symptoms, making early diagnosis challenging. The integration of histopathological features helps distinguish conditions with overlapping clinical presentations. For instance, psoriasis shows characteristic scalp and knee involvement, while lichen planus is associated with polygonal papules and oral mucosal involvement. Recognising these subtle patterns can reduce the need for invasive biopsies in early stages.

Suggestions for Early Identification by Doctors

- Priority should be given to patients presenting with persistent erythema and scaling that do not respond to initial treatment.
- To carefully evaluate lesion distribution, especially involvement of scalp, knees and elbows, which strongly indicate psoriosis.
- To record family history systematically, as genetic predisposition improves diagnostic confidence.
- To use histopathological features in combination rather than isolation to distinguish similar disease.
- Refer high risk or ambiguous cases for early machine learning assisted screening to avoid delayed daignosis.

Role of Machine Learning in early detection

- Machine learning models assist doctors by analysing complex patterns across multiple features simultaneously. These models reduce subjectively, improve diagnostic consistency, and support decision-making, particularly in cases with overlapping clinical characteristics. Early prediction using machine learning enables timely intervention, reduces diseases progression, and improves patient outcomes.